Lerneinheit C1: Rechnerarchitektur I - Schaltalgebra

Hallo und herzlich Willkommen zur Lerneinheit C1!

Wir haben bisher Algorithmen mit Hilfe von Scratch oder Java in Software implementiert. In der Lerneinheit C1 und C2 starten wir nun ein neues Thema und betrachten jetzt genauer, **wie wir Abläufe in kleinen Bauteilen, also konkret in Hardware, umsetzen können**. Dafür beschäftigen wir uns näher mit der Booleschen Algebra und logischen Ausdrücken, Wahrheitstabellen und Gatterschaltungen.

Jeder Computer kennt nur zwei Zustände: 0 oder 1. Etwas konkreter könnten wir auch von Strom an/aus sprechen. Oder mit Bezug auf eine "klassische" Festplatte sprechen wir von magnetisiert/nicht magnetisiert.

Wir haben bereits kennengelernt, wie wir mit diesen zwei Zuständen Zahlen und Buchstaben darstellen und rechnen können (Stichworte z.B.: Einer- und Zweierkomplement, ASCII-Codierung). In dieser Lerneinheit schauen wir uns die Hardware-Seite an und adressieren dabei die **folgenden drei Lernziele**:

- Sie können die wichtigsten **Begriffe zur Rechnerarchitektur** erklären und in Kontext zueinander bringen
- Sie können die **grundlegenden Darstellungsformen** (Wahrheitstabellen, Logische Ausdrücke und Gatterschaltungen) **erstellen**
- Sie können die **grundlegenden Darstellungsformen vereinfachen** und **ineinander überführen**

Literaturhinweise

Falls Sie sich fachwissenschaftlich näher mit dem Themen Rechnersysteme (Lerneinheiten 10&11) und Rechnernetzen (Lerneinheiten 12&13) beschäftigen möchten, empfehlen wir den **Band 2** der Informatik Reihe von Gumm & Sommer. Das Buch verwendet eine leicht andere Notation, lässt sich aber sehr gut lesen.

Gumm, Heinz-Peter and Sommer, Manfred. Rechnerarchitektur, Betriebssysteme, Rechnernetze, Berlin, Boston: De Gruyter Oldenbourg, 2017.
https://doi.org/10.1515/9783110442366

Autoren

Die Lerneinheit C1 wurde von **Antonella Bidlingmaier** und **Michael Blumenschein** erstellt und basiert in Auszügen auf Materialien von Dagmar Sorg und Christian Borgelt.

Viel Spass beim Kennenlernen der Schaltalgebra!

EINFÜHRUNG

Um was geht es?

GRUNDLAGEN

Einfache Funktionen am Beispiel von Schaltkreisen

Logische Operatoren und Schaltalgebra

Ein paar Worte zu Wahrheitstabellen

GLEICHUNGEN

Grundlagen für Gleichungen

Umwandlung einer Gleichung in eine Wahrheitstabelle

Umwandlung einer Wahrheitstabelle in eine Gleichung mit Hilfe der Disjunktiven Normalform

Äquivalenz mit Hilfe einer Wahrheitstabelle zeigen

DIGITALE LOGIK

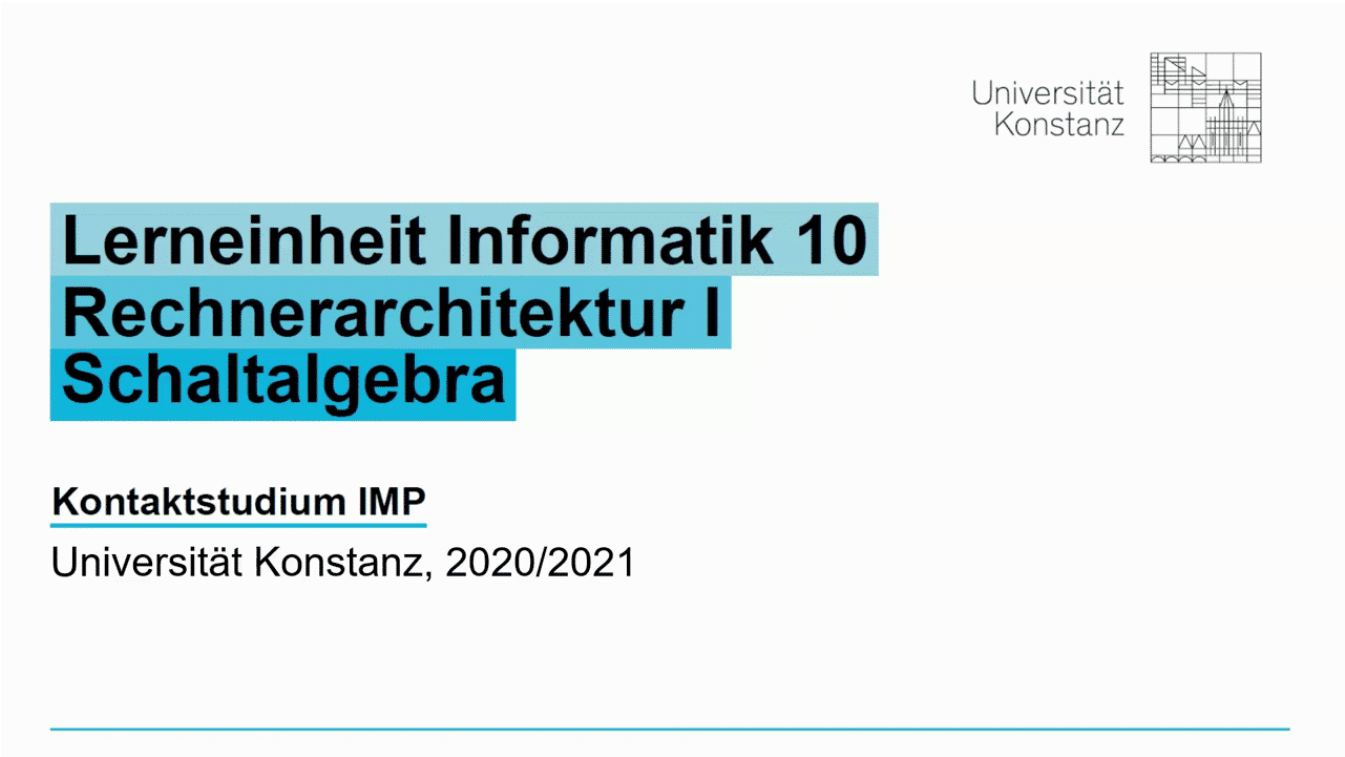
Gatterschaltungen

Axiomensystem - "Rechenregeln" für logische Ausdrücke

SCHLUSSWORT

Zusammenfassung

Um was geht es?



Die Funktion von Hardwarebauteilen lassen sich also mit **drei verschiedene Darstellungsformen** repräsentieren:

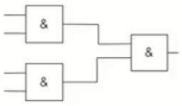
Wahrheitstabelle

a	b	z
0	0	0
0	1	0
1	0	0
1	1	1

Logischer Ausdruck

$$z = a \wedge b$$

Gatternetz = Implementierung

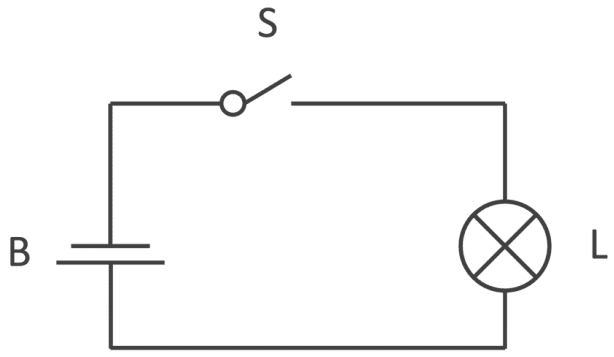


In den folgenden Lektionen werden wir lernen diese Darstellungsformen zu **verstehen**, zu **erstellen**, diese **umzuwandeln und zu vereinfachen** oder **ineinander zu überführen**.

Einfache Funktionen am Beispiel von Schaltkreisen

Im Folgenden erarbeiten wir uns anhand einiger Schaltkreisbeispiele die grundlegenden Funktionen von logischen Ausdrücken. Die Beispiele werden sich schrittweise erweitern und wir schauen uns neben den Schaltkreisen auch die entsprechende Wahrheitstabellen an.

Betrachten wir uns dafür zunächst einen einfachen Schaltkreis mit Batterie, Schalter und Lampe:



Schaltkreis

S	L
0	0
1	1

Wahrheitstabelle

Einfacher Stromkreis mit Wahrheitstabelle.

Dieser Schaltkreis (manchmal auch Schaltplan genannt) zeigt eine **Batterie B**, einen **Schalter S** und die **Lampe L**. Damit die Lampe leuchtet, (L=1) muss der Schalter S geschlossen (S=1) werden.

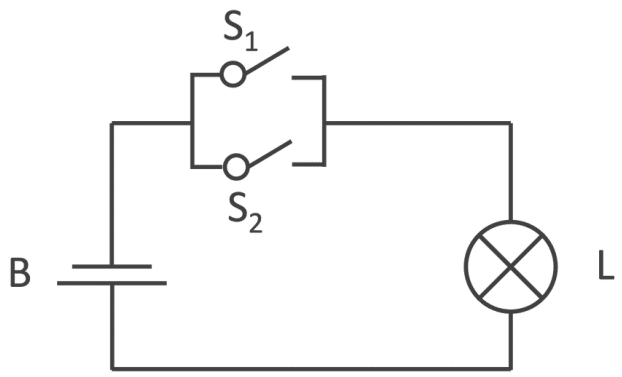
Die **Wahrheitstabelle** zeigt das Verhalten der Lampe, basierend auf den verschiedenen Zuständen des Schalters. Ist der Schalter geöffnet (S=0) ist auch die Lampe aus (L=0). Umgekehrt ist S = 1, gilt ebenfalls L = 1.

Die Wahrheitstabelle enthält also die **Variablen S und L** für die Bauteile unseres Schaltkreises. Da die Batterie immer angeschlossen ist und dadurch den Zustand nicht ändert (also immer Strom liefert), muss diese nicht notiert werden.

Was passiert nun, wenn wir mehrere Schalten verwenden?. Es gibt hierbei zwei grundlegende Kombinationsmöglichkeiten für mehrere Schalter in einem Schaltkreis: **Schalter werden nebeneinander (also parallel)** oder werden **nacheinander (also seriell bzw. in Reihe) angeordnet**. Wir sprechen hier entsprechend von der **Parallelschaltung** und **Serienschaltung**. Die Serienschaltung wird umgangsprachlich auch häufig **Reihenschaltung** genannt.

Parallelschaltung

Wenn man zwei Bestandteile (hier Schalter) nebeneinander mit einer Verzweigung schaltet, nennt man dies eine Parallelschaltung.



Parallelschaltung

S_1	S_2	L
0	0	0
0	1	1
1	0	1
1	1	1

Wahrheitstabelle

Parallelschaltung

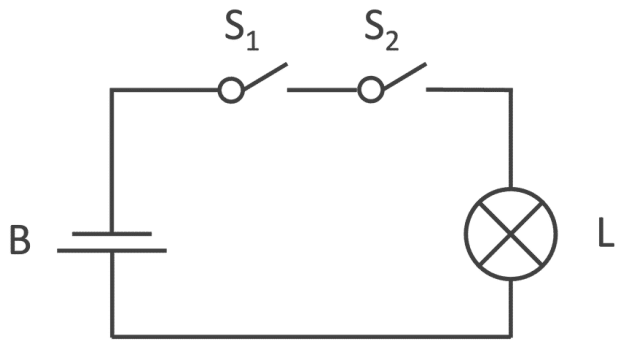
Damit die Lampe leuchtet muss **mindestens einer der beiden Schalter geschlossen sein**. Sind beide Schalter geschlossen leuchtet die Lampe ebenfalls.

Die Lampe ist nur dann aus (L=0) wenn beide Schalter geöffnet sind und der Stromkreis zwischen Batterie und Lampe vollständig unterbrochen ist.

Eine Parallelschaltung entspricht dem **logischen ODER** (schauen wir uns gleich noch näher an). Die Ergebnisvariable L ist genau dann 1, wenn mindestens einer der beiden Einhänge (also S1 oder S2) mit einer 1 belegt ist. Aus diesem Grund wird die Parallelschaltung auch als **Oder-Schaltung** bezeichnet.

Serienschaltung

Schaltet man zwei Bestandteile hintereinander (hier die Schalter S_1 und S_2) nennt man dies eine Serienschaltung. Häufig wird auch der Begriff Reihenschaltung verwendet.



Serienschaltung

S_1	S_2	L
0	0	0
0	1	0
1	0	0
1	1	1

Wahrheitstabelle

Serien- bzw. Reihenschaltung

Damit die Lampe leuchtet **müssen beide Schalter geschlossen sein**. Falls noch einer der beiden Schalter geöffnet ist, bleibt der Stromkreis unterbrochen und die Lampe bleibt aus.

Eine Serienschaltung entspricht dem **logischen UND** (schauen wir uns ebenfalls gleich noch näher an). Die Ergebnisvariable L ist nur dann 1, wenn S_1 **und** S_2 geschlossen sind. Aus diesem Grund wird die Serienschaltung ebenfalls als **Und-Schaltung** bezeichnet.

Logische Operatoren und Schaltalgebra

Mit der Serien- und Parallelschaltung haben wir bereits informell die logischen Operatoren **UND** und **ODER** kennengelernt. In diesem Kapitel werden wir diese und weitere Operatoren formalisieren und genauer anschauen.

Logische Operatoren (z.B. der UND-Operator) sind **mathematische Funktionen, die einen Wahrheitswert zurück geben.** Der UND-Operator liefert also die Wahrheitswerte

- wahr UND wahr = wahr
- wahr UND falsch = falsch
- falsch UND wahr = falsch
- falsch UND falsch = falsch

zurück. Um Schaltungen und Funktionen von (kleinen) Hardwarebauteilen zu beschrieben arbeiten wir im Folgenden aber nicht mit den Wahrheitswerten true und false. Sondern, **wir definieren true als 1 und false als 0**.

Die **Schaltalgebra definiert die Bedeutung der logischen Ausdrücke und definiert Regeln, wie wir mit logische Operatoren umgehen dürfen und wie wir mehrere Operatoren miteinander verbinden können.** Ein Teil der Schaltalgebra sind also Rechenregeln. Diese werden wir uns am Ende der Lerneinheit nochmals genauer anschauen.

Statt Schaltalgebra findet man auch häufig die Begriffe **boolesche Algebra** oder **boolesche Logik**. "Umgangssprachlich" und in vielen Büchern werde die drei Begriffe oft synonym verwendet. Formal ist die boolesche Algebra jedoch allgemein auf Wahrheitswerten definiert. Die Schaltalgebra ist im Gegensatz dazu eine spezielle Ausprägung der booleschen Algebra mit einer zweiwertigen Trägermenge; also 0 und 1. Alle Rechenregeln etc. sind aber identisch. Daher ist eine synonyme Verwendung durchaus vertretbar :).

Elementare logische Operatoren

Schauen wir uns jetzt die **drei elementaren logischen Operatoren UND, ODER und NICHT** einmal genauer an. Sehr häufig werden die **englische Bezeichnung (also AND, OR, NOT)** verwendet. Auch wir werden das in dieser Lerneinheit machen um die Operatoren von den Bindewörtern (und / oder / nicht) im Text besser zu unterscheiden.

AND (∧)

Der Operator AND beschreibt **unser umgangssprachliches 'UND'**. Nur wenn beide Teilaussagen (die Operanden a und b) wahr (true, 1) sind, ist die Aussage ebenfalls wahr.

Beispiel:
Tim geht zum See AND Lea fährt zum Park.

Die Aussage wird wahr wenn Tim zum See geht und Lea zeitgleich zum Park fährt.

a	b	**a ∧ b**
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

OR (∨)

Der OR Operator gibt den Wahrheitswert 1, wenn mindestens ein Operatoren den Wert 1 hat.

Im Gegensatz zu dem AND kann man das OR **nicht** mit dem umgangssprachlichen (Entweder-)ODER gleichsetzten. **Auch wenn beide Operatoren wahr sind, wird der Ausdruck a ∨ b wahr.**

Beispiel:
Tim geht zum See OR Lea fährt zum Park.

Die Aussage wird wahr, wenn

- Tim zum See geht
- Lea zum Park fährt
- oder wenn Tim zum See geht und Lea zeitgleich zum Park fährt.

a	b	**a ∨ b**
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

Die Notation von AND und OR sind sich sehr ähnlich und werden häufig von Schüler*innen verwechselt. Hier kann vielleicht folgende **Eselsbrücke** helfen:

- AND ∧ : stellen Sie sich ein Strichmännchen vor bei dem der AND Operator ∧ die Beine repräsentiert. Das Männchen steht gleichzeitig auf dem linken **und** auf dem rechten Bein/Fuß.

- OR ∨ : das kleine v des OR Operators fällt nach links **oder** nach rechts, wenn man es nicht "festhält".

NOT (¬)

Der Operator NOT (im dt. NICHT) **verneint eine Aussage.** Mit anderen Worten, der Wahrheitswert einer Aussage wird negiert, also umgedreht.

a	**¬a**
0	**1**
1	**0**

Beispiel:
NOT Tim geht zum See

Diese Aussage wird wahr, wenn Tim nicht zum See geht (sondern z.B. in die Stadt).

Der Operator NOT wird häufig in Kombination mit anderen Operatoren verwendet und kann auch geschachtelt werden. Dazu gleich noch mehr.

Beispiel:
Liam geht nicht zum See oder auch nicht zum Park.
¬ See ∨ **(¬ Park)** ergibt den korrekten Wahrheitswert für dieses Szenario.

Nun kamen dem ein oder anderen die Wahrheitstabellen vielleicht schon bekannt vor. Wir erinnern uns an die Wahrheitstabellen der Serien- und Parallenschaltung zurück. Die **Serienschaltung zeigt die gleiche Tabelle wie das logische AND,**

die Parallelschaltung ist der des logischen OR gleich. Damit können wir nun Schaltkreise auch mathematisch mit einem logischen Operator dazustellen. Die mathematische Darstellung ist wichtig, damit wir mit dieser rechnen können, ohne sich immer bildlich den Stromkreis vorstellen zu müssen, oder ihn mit Wahrheitstafeln zu notieren.

Wie wir im Laufe dieser Lerneinheit noch sehen werden, können wir jede komplexe Schaltung mit Hilfe der drei elementaren logischen Operatoren AND, OR und NOT darstellen. Weitere Operatoren sind nicht nötig.

Weitere logische Operatoren

Auch wenn wir jede Funktion mit AND, OR und NOT darstellen können, ist dies teilweise sehr umständlich. Es gibt weitere Operatoren, die uns bei der Erstellung und Beschreibung von Schaltungen behilflich sind. Diese werden im Folgenden vorstellt:

XOR (⊕)

Das XOR ist unser umgangssprachliches ENTWEDER-ODER. Die Aussage a ⊕ b wird wahr, wenn e**ine der beiden Teilaussagen a oder b wahr ist**. Sind beide Aussagen a und b wahr dann ist a ⊕ b falsch.

a	b	**a ⊕ b**
0	0	**0**
0	1	**1**
1	0	**1**

1	1	**0**

NAND ($\overline{\wedge}$)

Das NAND **entspricht der Kombination NOT + AND.** Die Aussage a $\overline{\wedge}$ b negiert also die Aussage des klassischen AND Operators. **a $\overline{\wedge}$ b ist also 1, solange mindestens einer der beiden Variablen 0 ist.** Sind beide Eingänge 1, so ist die Aussage 0.

a	b	**a $\overline{\wedge}$ b**
0	0	**1**
0	1	**1**
1	0	**1**
1	1	**0**

NOR ($\overline{\vee}$)

Analog zum NAND entspricht das NOR **der Kombination von NOT + OR**. Die Aussage a $\overline{\vee}$ b negiert also die Aussage des klassischen OR Operators. **a $\overline{\vee}$ b ist also nur 1, sofern a = b = 0 gilt.** In allen anderen Kombinationen liefert a $\overline{\vee}$ b eine 0.

a	b	**a $\overline{\vee}$ b**
0	0	**1**
0	1	**0**

1	0	**0**
1	1	**0**

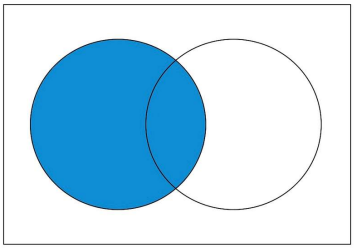
XNOR (⊙)

Das XNOR Operator **negiert die Aussagen des XOR Operators**. Die Aussage **a ⊙ b wird wahr, wenn beide Variablen den gleichen Wert haben**; also wenn gilt a = b = 0 oder a = b = 1. Unterscheiden sich die Wahrheitswerte von a und b liefert a ⊙ b den Wahrheitswert 0.

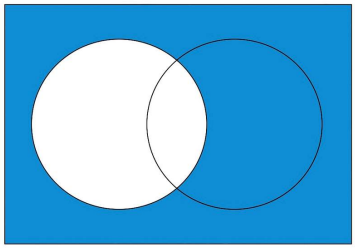
a	b	**a ⊙ b**
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**1**

Darstellung im Venn-Diagramm

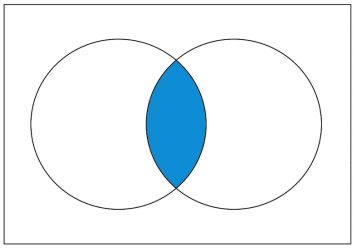
Neben der Darstellung in Wahrheitstabellen **können Operatoren ebenfalls in Venn-Diagrammen veranschaulicht** werden. Hier ein paar Beispiele:



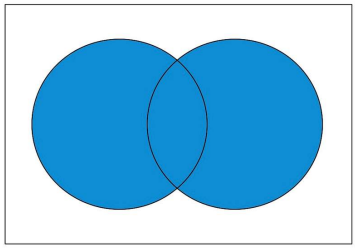
Operand **A**



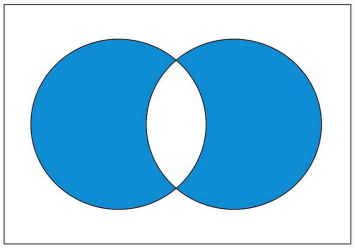
¬A



A ∧ B



A ∨ B



A ⊕ B

Notationen und Schreibweisen

Es gibt ganz unterschiedliche Schreibweisen bzw. Notationen für die logischen Operatoren. Je nach Schulart kann ggfs. auch eine bestimmte Schreibweise vorgegeben sein.

Damit Sie auch weitere Literatur verstehen und wir einheitlich in diesem Kurs bezüglich unserer Notation bleiben, betrachten wir nun, welche anderen Formen der Notation es gibt - und welche Notationen ähnlich sind, aber etwas anderes bedeuten.

Wir werden im gesamten Kontaktstudium die in blau hervorgehobene Schreibweise verwenden. Bitte verwenden Sie auch diese für Ihre Übungen und in der Klausur.

Eine der häufigsten Notationen für die AND / OR Operatoren ist der Malpunkt (für AND) bzw. das Plus Zeichen (für OR); also

- **a · b** statt **a ∧ b**
- **a + b** statt **a ∨ b**

Hintergrund: Bei den Grundrechenarten gilt Punkt-vor-Strich. Wir werden das gleich noch im Detail sehen - auch bei den logischen Operatoren gibt es Rechenregeln. Auch hier gilt: das AND bindet stärker als das OR und daher gilt auch hier die "übertragene" Punkt-vor-Strich Regel.

Wir werden aber bei der oben verwendeten Schreibweise bleiben:

UND: ∧ **ODER: ∨**

Eine ähnliche Notation mit einem zentralen inhaltlichen Unterschied ist folgende Schreibweise:

Schnittmenge: ∩ Vereinigungsmenge: ∪

Diese sind bekannt aus der Mengenlehre und werden auch bei den obigen Venn-Diagrammen in der Mengenlehre mit ihrer Schreibweise zu verwenden. Es ist wichtig hier zu differenzieren, **dass wir mit Variablen arbeiten und nicht mit Mengen**.

Betrachtet man z.B. die Definition von der Schnittmenge $A \cap B = \{x : x \in A \wedge x \in B\}$, so sieht man, dass diese Symbole nicht äquivalent verwendet werden können!

Die das NOT Operationszeichen kann ebenfalls mehrere nicht identische Notationen enthalten:

NICHT: ¬ NICHT: Ein Strich über dem A (A quer)

Dies sind die am weitest verbreiteten Schreibweisen der boolschen Algebra und korrespondierenden Mengenlehre. In diesem Kurs jedoch verwenden wir fortan die in blau markierten Schreibweisen.

- UND: ∧
- ODER: ∨
- NICHT: ¬
- XOR: ⊕

Weitere Schreibweisen finden Sie u.a. hier:
https://de.wikipedia.org/wiki/Boolesche_Algebra -> Sektion Schreibweisen.

Ein paar Worte zu Wahrheitstabellen

Wir haben Wahrheitstabellen bereits an mehreren kleinen Beispielen kennengelernt. Im Folgenden finden Sie nochmals ein paar wichtige Hinweise und Rahmenbedingungen.

Bezeichnungen
Wahrheitstabellen werden häufig auch als **Wahrheitstafel** oder als **Schalttabelle** bzw. **Schalttafel** bezeichnet. Gerne können Sie alle Begriffe synonym verwenden.

Wahrheitswerte
Die Variablen in den Wahrheitstabellen nehmen die 0 und 1 an und repräsentieren damit die Werte true/false bzw. wahr/falsch. Häufig sieht man in Büchern auch die Schreibweise mit true/false bzw. die Kurzschreibweisen

- t für true
- f für false

Ziel einer Wahrheitstabelle
Zentrales Ziel einer Wahrheitstabelle ist es die Wahrheitswerte aller Variablen sowie einer (komplexen) Aussage zu notieren.

Angenommen wir möchten die Wahrheitstabelle zu folgendem logischen Ausdruck erstellen: $a \wedge b \wedge c = z$. Ziel ist es dann für jede Kombination von Wahrheitswerten für die Variablen a, b und c den Wahrheitswert für z zu definieren.

Wir legen dazu eine entsprechende Tabelle an.

- **Die Anzahl der Spalten** entspricht der Anzahl der Variablen, inkl. der Zielvariablen z - also in unserem Fall 4.
- **Die Anzahl der Zeilen** entspricht allen Kombinationsmöglichkeiten von 0en und 1sen für die Variablen a, b und c. In unserem Fall also 2^3 = 8 Zeilen.

Um **alle Kombinationsmöglichkeiten** für a, b und c strukturiert abzubilden, bietet es sich an, hier **binär zu zählen**. Damit stellen wir sicher, dass wir jede Kombinationsmöglichkeit abgebildet haben. Wir starten also mit

- 0 0 0 (jeweils für a, b, c)
- 0 0 1
- 0 1 0
- 0 1 1
- ...
- 1 1 0
- 1 1 1 (jeweils für a, b, c)

Schließlich können wir in der Spalte z die Wahrheitswerte für den logischen Ausdruck eintragen. Für $a \wedge b \wedge c = z$ gilt, dass nur in der letzten Zeile ($a = b = c = 1$) gilt $z = 1$.

a	b	c	**z**
0	0	0	**0**
0	0	1	**0**
0	1	0	**0**

0	1	1	**0**
1	0	0	**0**
1	0	1	**0**
1	1	0	**0**
1	1	1	**1**

Grundlagen für Gleichungen

Komplexe Schaltkreise enthalten viele Verzweigungen. Um diese mathematisch darstellen zu können werden mehrere logische Operatoren benötigt. Diese können einfach aneinander gereiht werden, solange deren Teilaussage schlüssig ist.

Beispiel:

richtig: $a \vee b \wedge c$

richtig: $a \vee \neg b$

falsch: $a \vee \wedge b$

Diese Verkettung von Aussagen werden wie in der Mathematik Gleichungen genannt. Wie der Begriff bereits sagt, wollen wir Aussagen über Gleichheiten tätigen können. Dafür benötigen wir eine Operatorrangfolge. Ein bekanntes Beispiel aus der Mathematik ist die Punkt-vor-Strich Regel.

Operatorrangfolge

Betrachten wir die Operatorrangfolge der Mathematik lassen sich Parallelen erkennen. Beim Auswerten einer Gleichung gilt die folgende Reihenfolge der Operatoren:

1. NOT ¬
2. AND ∧
3. OR ∨

D.h. das NOT bindet stärker als das AND - und AND stärker als OR. Ganz ähnlich zur "normalen Mathematik" bei der zuerst Vorzeichen, dann Punkt, dann Strich gerechnet wird.

Beispiel
Gegeben ist folgende Gleichung: $a \vee \neg b \wedge c$. Wenn wir die Auswertungsreihenfolge mit Klammern verdeutlichen erhalten wir $a \vee ((\neg b) \wedge c)$ - d.h. zuerst NOT b; dann ein AND mit c und das Ergebnis wird dann OR a gerechnet.

Klammern

Mit Klammern können wir das Auswerten der Operatorrangfolge beeinflussen. Auch geschachtelte Klammern sind möglich. Wie wir aus der Mathematik kennen, arbeiten wir uns bei der Auswertung dabei "von innen nach außen" vor. Das folgende Beispiel zeigt, wie wir mit Hilfe von Klammern und einer Kombinationen von Operatoren einen komplexen logischen Ausdruck erzeugen können.

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = z$$

z repräsentiert hierbei das Ergebnis der Gleichung. Das könnte, beispielsweise das Brennen einer Lampe innerhalb eines Schaltkreises sein.

In den nächsten Lektionen lernen wir nun mit solchen (komplexen) Gleichungen umzugehen und diese in Wahrheitstabellen und Gatterschaltungen zu überführen.

Umwandlung einer Gleichung in eine Wahrheitstabelle

Um eine Gleichung zu verstehen und im Anschluss auch zu vereinfachen können wir den entsprechenden logischen Ausdruck in eine Wahrheitstabelle übersetzen. Als Beispiel schauen wir uns dazu die bereits bekannte Gleichung aus dem letzten Kapitel näher an:

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$$

In folgendem Video führen wir die einzelnen Schritte nacheinander aus:

Gehen wir den Prozess noch einmal Schritt für Schritt durch:

Anzahl der Variablen

Welche Variablen und deren Namen kommen in unserer Gleichung vor. Notiere nur Variablen die auch Teil der Gleichung sind!

a	b	c

Anzahl der Reihen

Hierbei müssen wir alle Kombinationen von Variablenwerten notieren. Von hier an wird die Tabelle nur noch in Spalten Richtung wachsen. Wir füllen die Tabelle indem wir binär zählen.

a	b	c
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0

1	0	1
1	1	0
1	1	1

Nächste Spalte

Nun betrachten wir erneut die Gleichung $(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$.
Laut Operatorrangfolge wissen wir, dass die Klammersetzung an höchster Stelle steht. Betrachtet man die Gleichung von vorne nach hinten, wird der Teil $(\neg (a \wedge b) \wedge \neg (a \vee c))$ als erstes gelöst. Zuerst sind die innersten Klammern in diesem Teil zu berechnen.

$(a \wedge b)$:
Dieser Teil beinhaltet zwei Variablen a und b, wir betrachten die beiden Spalten und deren Inhalt. Die Lösung unseren neuer Spalte kommt zustande indem wir die Operation $\wedge$ (AND) auf die entsprechenden Werte in den Zeilen anwenden.

1ste Zeile:
a = 0; b = 0;
a $\wedge$ b = 0 $\wedge$ 0 **= 0**
Diese 0 tragen wir nun in unsere neue Spalte in der ersten Zeile der Werte ein.

Für die folgenden Zeilen ist der Vorgang identisch, wir betrachten die Werte der grau makierten Spalten um unseren Wert für $(a \wedge b)$ zu berechnen.

a	b	c	$a \wedge b$
0	0	0	**0**
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Umbenennen und nächste Spalte

Die Gleichung $(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$ enhält eine Negation direkt vor den Klammern. Basierend auf Operatorrangfolge muss dies betrachtet werden bevor wir später damit weiter rechnen können. Deshalb betrachten wir direkt die Negation.

Damit nicht $\neg (a \wedge b)$ notiert werden muss, geben wir dem Term $(a \wedge b)$ einen neuen Namen **z** gegeben. Dies wird für die Übersichtlichkeit getan, ist jedoch kein muss. Beachtet werden muss, dass der Variablenname nicht bereits in der ursprünglichen Gleichung existiert. Wir passen unsere Gleichung entsprechend an, oder notieren die Unterschiede, damit wir den Überblick auf dieser Seite nicht verlieren.

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = (\neg \mathbf{z} \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$$

Gibt es den Teil $(a \wedge b)$ mehrfach in der Gleichung, wird auch dieser Teil direkt mit einem **z** ersetzt!

Nun zu $\neg$ z:
Wir benötigen keiner der ersten drei Variablen (a, b, c), da nur z Teil des Ausdrucks ist. Nehmen wir Spalte vier zur Hand und negieren all seine Werte, besitzen wir die Werte für Spalte fünf.

Da die Gleichung immer weiter entschlüsselt wird und wir somit diesen Teil zwangsweise erneut benötigen, geben wir auch diesem einen neuen Namen.

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = (\mathbf{y} \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$$

a	b	c	$z = a \wedge b$	$y = \neg z$
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1

1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	0

Nächste Spalten

Fügen wir direkt zwei Spalten auf einmal hinzu. Erneut wollen wir $(y \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$ einen Teil $(a \vee c)$ und seine Negation $\neg (a \vee c)$ betrachten.

x = $(a \vee c)$:
Hier werden nun die Spalten für die Variablen b und c betrachtet und diesmal mit dem logischen Operator $\vee$ (oder) verbunden.

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = (y \wedge \neg \mathbf{x}) \wedge (\neg b \vee b)$$

w = ¬ x:
Erneut benötigen wir hier nur die Spalte für Variable x und betrachten deren Negation (¬).

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = (y \wedge \mathbf{w}) \wedge (\neg b \vee b)$$

a	b	c	$z = a \wedge b$	$y = \neg z$	$x = (a \vee c)$	$w = \neg x$
0	0	0	0	1	0	1
0	0	1	0	1	1	0
0	1	0	0	1	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	0	1	0	1	1	0
1	1	0	1	0	1	0
1	1	1	1	0	1	0

Ersten Teil der Gleichung lösen

Der erste Teil der Gleichung ist fast gelöst. Es gfehlt nur noch $y \wedge w$.

$v = y \wedge w$:

Zu betrachten sind die Spalten der Variablen y und w.

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = (\mathbf{v}) \wedge (\neg b \vee b)$$

a	b	c	z = $a \wedge b$	y = $\neg$ z	x = $(a \vee c)$	w = $\neg$ x	v = y $\wedge$ w
0	0	0	0	1	0	1	1
0	0	1	0	1	1	0	0
0	1	0	0	1	0	1	1
0	1	1	0	1	1	0	0
1	0	0	0	1	1	0	0
1	0	1	0	1	1	0	0
1	1	0	1	0	1	0	0
1	1	1	1	0	1	0	0

Hinterer Teil der Gleichung

Je größer die Tabelle desto unübersichtlicher. Dank der Variablennamen müssen wir jedoch nur die wenigen Spalten betrachten die für uns relevant sind. Da wir nun

im zweiten Teil der Gleichung angelangt sind werden wir erneut die urspünglichen Spalten betrachten müssen.

u = $\neg b$:

Wir gehen zurück zur zweiten Spalte, für die Variablenwerte von b.

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = (\vee) \wedge (\mathbf{u} \vee b)$$

t = u $\vee$ b:

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = \vee \wedge \mathbf{t}$$

a	b	c	z = $a \wedge b$	y = $\neg$ z	x = ($a \vee c$)	w = $\neg$ x	v = y $\wedge$ w	u = $\neg b$	t = u $\vee$ b
0	0	0	0	1	0	1	1	1	1
0	0	1	0	1	1	0	0	1	1
0	1	0	0	1	0	1	1	0	1
0	1	1	0	1	1	0	0	0	1
1	0	0	0	1	1	0	0	1	1
1	0	1	0	1	1	0	0	1	1
1	1	0	1	0	1	0	0	0	1

1	1	1	1	0	1	0	0	0	1

Lösung der Gleichung

Zu diesem Zeitpunkt müsste das Vorgehen bereits klar sein :)
Wir fügen die letzten beiden Teile der Gleichung zusammen.

s = v ∧ t:

$$(\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b) = \mathbf{s}$$

a	b	c	z = $a \wedge b$	y = ¬ z	x = $(a \vee c)$	w = ¬ x	v = y ∧ w	u = $\neg b$	t = u ∨ b	s = v ∧ t
0	0	0	0	1	0	1	1	1	1	1
0	0	1	0	1	1	0	0	1	1	0
0	1	0	0	1	0	1	1	0	1	1
0	1	1	0	1	1	0	0	0	1	0

1	0	0	0	1	1	0	0	1	1	0
1	0	1	0	1	1	0	0	1	1	0
1	1	0	1	0	1	0	0	0	1	0
1	1	1	1	0	1	0	0	0	1	0

Damit haben wir die Lösung unserer Gleichung. Betrachten wir nur die letzte Zeile so sieht der Weg dorthin gar nicht so lange aus...

a	b	c	...	$s = (\neg(a \wedge b) \wedge \neg(a \vee c)) \wedge (\neg b \vee b)$
0	0	0	...	1
0	0	1	...	0
0	1	0	...	1
0	1	1	...	0
1	0	0	...	0
1	0	1	...	0
1	1	0	...	0

Damit haben wir die Lösung unserer Gleichung. Betrachten wir nur die letzte Zeile so sieht der Weg dorthin gar nicht so lange aus...

a	b	c	...	$s = (\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$
0	0	0	...	1
0	0	1	...	0
0	1	0	...	1
0	1	1	...	0
1	0	0	...	0
1	0	1	...	0
1	1	0	...	0
1	1	1	...	0

Umwandlung einer Wahrheitstabelle in eine Gleichung mit Hilfe der Disjunktiven Normalform

Nachdem wir nun einen logischen Ausdruck in eine Wahrheitstabelle überführt haben, schauen wir uns nun die umgekehrte Reihenfolge an: Umwandlung einer Wahrheitstabelle in eine Gleichung. Wir leiten dazu die **Disjunktive Normalform** der Wahrheitstabelle ab.

Wir schauen uns wieder zuerst den Ablauf in einem kurzen Video an und gehen anschließend nochmals ausführlich auf die einzelnen Schritte ein.

Handling von Wahrheitstabellen

Vereinfachen Sie folgenden logischen Ausdruck

$$\underbrace{\big(\neg(a \wedge b) \wedge \neg(a \vee c)\big) \wedge (\neg b \vee b)}_{= t}$$

a	b	c	...	$t = v \wedge u$
0	0	0	...	1
0	0	1	...	0
0	1	0	...	1
0	1	1	...	0
1	0	0	...	0
1	0	1	...	0
1	1	0	...	0
1	1	1	...	0

Formen der Konjunktionen

Zur Erstellung der Disjunktiven Normalform werden nur die Spalten der Wahrheitstafel für die Variablen und die Spalte für die Wahrheitswerte der gesamten Funktion (letzte Spalte) benötigt. Weiterhin betrachten wir nur die Zeilen in denen der Funktionswert 1 ist.

Anschließend erstellt man für jede der markierten Zeilen die Konjunktion (ver-unden). Hat eine Variable dabei den Wert 0 muss diese negiert (ver-neint) werden.

a	b	c	...	s = (¬ $(a \wedge b)$	Konjunktion

				$\wedge \neg (a \vee c)) \wedge (\neg b \vee b)$	
0	0	0	...	1	$\neg a \wedge \neg b \wedge \neg c$
0	0	1	...	0	-
0	1	0	...	1	$\neg a \wedge b \wedge \neg c$
0	1	1	...	0	-
1	0	0	...	0	-
1	0	1	...	0	-
1	1	0	...	0	-
1	1	1	...	0	-

Ableitung Disjunktive Normalform

Bildet man nun die Disjunktion (ver-odern) über alle entstandenen Konjunktionen erhält man die Disjunktive Normalform.

$$D = (\neg a \wedge \neg b \wedge \neg c) \vee (\neg a \wedge b \wedge \neg c)$$

Die disjunktive Normalform ist äquivalent zu der ursprünglichen Gleichung, da diese daraus hevorgeht. **In späterer Lektion** lernen wir das vereinfachen von Ausdrücken, welche anschließend zu dem erstellen dieser Form erfolgt. Daraus würde sich folgendes ergeben:

$$(\neg a \wedge \neg b \wedge \neg c) \vee (\neg a \wedge b \wedge \neg c) \;\; = \;\; \neg a \wedge \neg c$$

und somit:

$$(\neg\,(a \wedge b) \wedge \neg\,(a \vee c)) \wedge (\neg b \vee b) \;\; = \;\; (\neg a \wedge \neg b \wedge \neg c) \vee (\neg a \wedge b \wedge \neg c) \;\; = \;\; \boldsymbol{\neg a \wedge \neg c}$$

Äquivalenz mit Hilfe einer Wahrheitstabelle zeigen

In vielen Scenarien müssen wir überprüfen, ob logische Ausdrücke äquivalent zueinander sind; z.B. ob ein komplexer und ein einfacher logischer Ausdruck identische Wahrheitswerte für alle Eingaben liefern. Um das zu machen haben wir grundsätzlich zwei Möglichkeiten:

- Möglichkeit 1: Wir versuchen mit Hilfe des Axiomensystems (also den Rechenregeln für der booleschen Algebra) einen Ausdruck einen einen anderen zu überführen. Das schauen wir uns später an.
- Möglichkeit 2: Wir überführen beide Ausdrücke in jeweils eine Wahrheitstabelle und überprüfen ob die entsprechenden Wahrheitswerte in allen Zeilen der Tabelle identisch sind.

Schauen wir uns Möglichkeit 2 anhand des folgenden Beispiels genauer an:

Wir wollen die semantische Äquivalenz der folgenden logischen Ausdrücke zeigen.

$$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$$

Wir starten wieder mit einem Video und gehen anschließend nochmals auf die einzelnen Schritte ein:

Handling von Wahrheitstabellen

Zeigen Sie die semantische Äquivalenz der folgenden logischen Ausdrücke:

$$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$$

a	b	c					
0	0	0					
0	0	1					
0	1	0					
0	1	1					
1	0	0					
1	0	1					
1	1	0					
1	1	1					

Wahrheitstafel der linken Seite der Gleichung

Wie wir bereits im Umgang mit Wahrheitstabellen gelernt haben, betrachten wir die Klammersetzung und arbeiten uns von innen nach außen.

$$a \vee z = (a \vee b) \wedge (a \vee c)$$

a	b	c	$z = b \wedge c$
0	0	0	0

a	b	c	$z = b \wedge c$
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$$y = (a \vee b) \wedge (a \vee c)$$

a	b	c	$z = b \wedge c$	$y = a \vee z$
0	0	0	0	**0**
0	0	1	0	**0**
0	1	0	0	**0**
0	1	1	1	**1**
1	0	0	0	**1**

a	b	c	$z = b \wedge c$	$y = a \vee z$
1	0	1	0	**1**
1	1	0	0	**1**
1	1	1	1	**1**

Wir markieren die letzte Spalte, da dies unser Ergebnis der linken Gleichung ist, welches wir später vergleichen wollen.

Wahrheitstafel der rechten Seite der Gleichung

$$y = x \wedge (a \vee c)$$

a	b	c	$z = b \wedge c$	$y = a \vee z$	$x = a \vee b$
0	0	0	0	**0**	0
0	0	1	0	**0**	0
0	1	0	0	**0**	1
0	1	1	1	**1**	1

a	b	c	$z = b \wedge c$	$y = a \vee z$	$x = a \vee b$
1	0	0	0	**1**	1
1	0	1	0	**1**	1
1	1	0	0	**1**	1
1	1	1	1	**1**	1

$y = x \wedge w$

a	b	c	$z = b \wedge c$	$y = a \vee z$	$x = a \vee b$	$w = a \vee c$
0	0	0	0	**0**	0	0
0	0	1	0	**0**	0	1
0	1	0	0	**0**	1	0
0	1	1	1	**1**	1	1
1	0	0	0	**1**	1	1
1	0	1	0	**1**	1	1
1	1	0	0	**1**	1	1

a	b	c	$z = b \wedge c$	$y = a \vee z$	$x = a \vee b$	$w = a \vee c$
1	1	1	1	**1**	1	1

$y = v$

a	b	c	$z = b \wedge c$	$y = a \vee z$	$x = a \vee b$	$w = a \vee c$	$v = x \wedge w$
0	0	0	0	**0**	0	0	**0**
0	0	1	0	**0**	0	1	**0**
0	1	0	0	**0**	1	0	**0**
0	1	1	1	**1**	1	1	**1**
1	0	0	0	**1**	1	1	**1**
1	0	1	0	**1**	1	1	**1**
1	1	0	0	**1**	1	1	**1**
1	1	1	1	**1**	1	1	**1**

Wir markieren erneut die letzte Spalte, da diese das Ergebnis der rechten Seite der Gleichung repräsentiert.

Äquivalenz prüfen

Wir haben nun die Tabelle mit unseren zwei markierten Spalten. Um die Äquivalenz zu prüfen, müssen nur die beiden Spalten vergleichen werden. Sind die beiden Spalten identisch, so ist die Äquivalenz gegeben, anderenfalls nicht.

a	b	c	$z = b \wedge c$	$y = a \vee z$	$x = a \vee b$	$w = a \vee c$	$v = x \wedge w$
0	0	0	0	**0**	0	0	**0**
0	0	1	0	**0**	0	1	**0**
0	1	0	0	**0**	1	0	**0**
0	1	1	1	**1**	1	1	**1**
1	0	0	0	**1**	1	1	**1**
1	0	1	0	**1**	1	1	**1**
1	1	0	0	**1**	1	1	**1**
1	1	1	1	**1**	1	1	**1**

Wichtig ist hier korrekt zu markieren, welche Spalten relevant sind um die Äquivalenz zu zeigen (-> siehe fett-markierte Spalten y und v). Diese Spalten sind genau die vorher markierten finalen Ausdrücke der beiden Seiten.

Gatterschaltungen

Zur Realisierung eines logischen Ausdrucks nehmen wir uns die Gatterlogik zur Hand. In der Gatterlogik gibt es für jeden logischen Ausdruck (also z.B. AND, OR, NOT) ein sogenanntes **Gatter** - ein kleines Bauteil, das so auch ganz praktisch in Hardware implementiert werden kann.

Ziel ist es also einen logischen Ausdruck in eine Gatterschaltung (auch Gatternetz genannt) zu überführen. Eine Gatterschaltung entspricht dabei einem Schaltplan mit speziellen Zeichen statt der üblichen Transistoren, Dioden und Wiederständen (vgl. Physik-Lerneinheiten) um die Pläne zu vereinfachen. In den Gatterschaltungen werden auch keine Spannungen o.ä. angelegt, sondern wir arbeiten weiterhin mit Wahrheitswerten bzw. 0 und 1. In einer konkreten Hardware Implementierung würde dann eine 1 einer konkreten Spannung von z.B. 5V bedeuten.

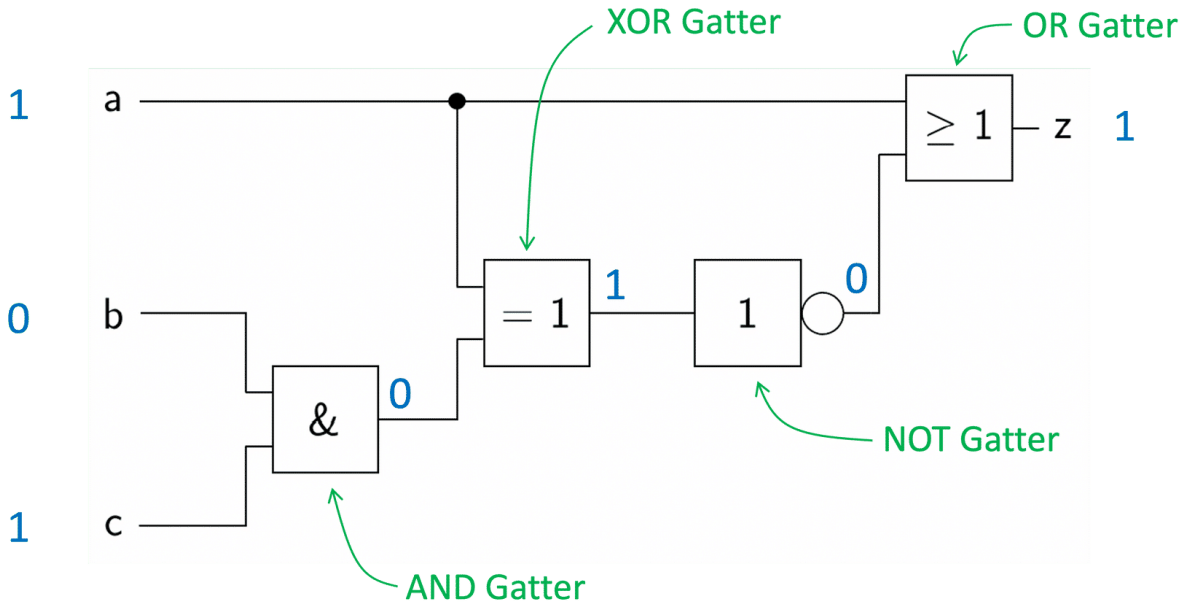
Die Variablen werden zu Eingangssignalen und das Ergebnis eines Ausdrucks zum Ausgangssignal.

Eine einfache Gatterschaltung mit den drei Eingangssignalen a, b und c sowie dem Ausgangssignal z könnte z.B. so aussehen.

Stellen Sie sich die Linien dabei als Kabel oder Leitungen vor, die alle Eingänge, Ausgänge und Gatter miteinander verbinden.

Beispielhaft sind die Eingangssignale a=1, b=0 und c=1 an das Gatternetz angeschlossen. Schauen wir uns von links nach rechts die jeweiligen Gatter an, so können wir die

Eingangssignale bis zum Ausgang z propagieren (siehe Wahrheitswerte in blau). Für die gegebene Eingabe erhalten wir also die Ausgabe z = 1.

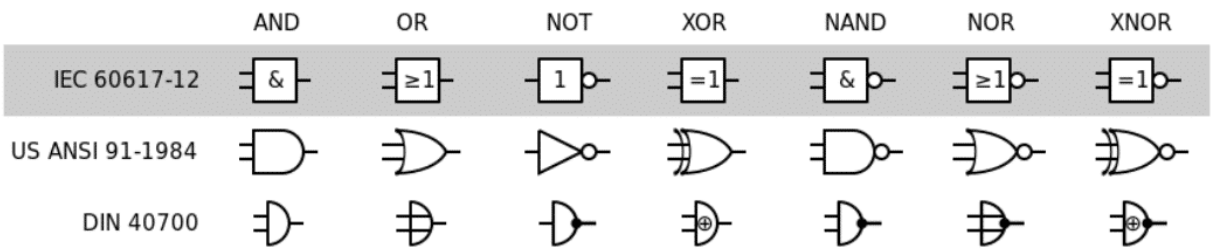


Beispiel: Gatterschaltung mit konkreten Eingangssignalen

Schreibweisen

Auch in der Gatterlogik gibt es verschiedene Schreibweisen bzw. Normen um Gatterzeichen darzustellen. Im europäischen Raum wird meist die Norm IEC 60617-12 verwendet. IEC steht dabei für International Electronic Commission. In Schaltplänen findet man auch recht häufig die Norm US ANSI 91-1984. Die Darstellung nach DIN 40700 ist veraltet und wird eigentlich nicht mehr verwendet.

Im Kontaktstudium verwenden wir die IEC 60617-12 Norm.



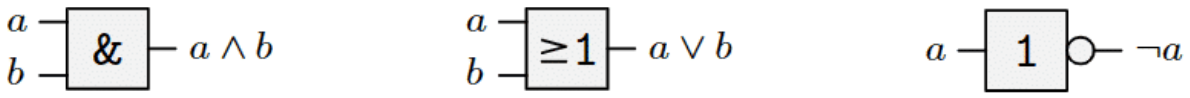
Verschiedene Normen für Gatterzeichen; NAND, NOR und XNOR haben wir bisher noch nicht kennengelernt.

Elementare Gatter

Es gibt drei elementare Gatter in der Gatterlogik: AND, OR und NOT. Mit Hilfe dieser drei Gattern können alle Schaltungen erstellt werden und auch die komplexeren Gatter (XOR, NAND, NOR und XNOR) implementiert werden.

Die meisten Gatter besitzen zwei Eingänge und ein Ausgang, basierend auf wie viele Variablen der logische Operator verbindet. Die Negation bildet die Ausnahme. Da der logische Operator nur an einer Variablen hängt, wird nur ein Eingang benötigt.

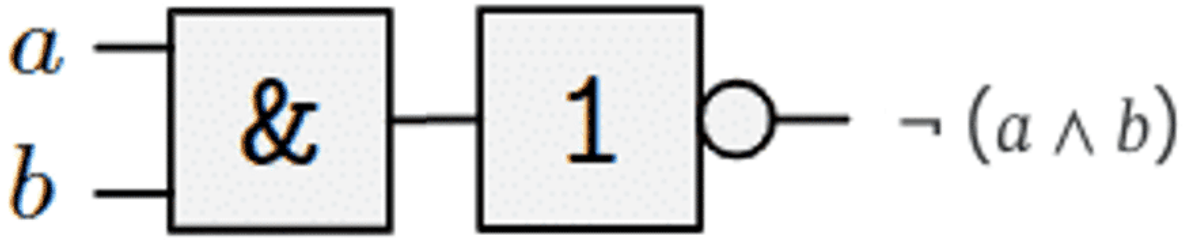
Wichtig: es dürfen nicht einfach zusätzliche Eingänge hinzugefügt werden; ein UND Gatter mit drei Eingängen gibt es also nicht.



Ein- und Ausgänge der drei elementaren Gatter.

Haben wir zusammengesetzte Ausdrücke, z.B. $\neg(a \wedge b)$, so müssen wir auch mehrere Gatter verwenden. Wir beginnen innen bei dem Gatter des AND und folgen dann mit dem der

Negation.



Gatterschaltung des logischen Ausdrucks $\neg (a \wedge b)$

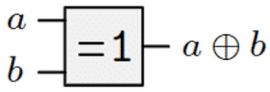
Zusammengesetze Gatter

Am Beispiel des XORs betrachten wir wie eine Implementierung von nicht elementaren Gattern funktioniert. Nicht elementare Gatter sind Gatter die aus mehr als einem Element bestehen, jedoch so häufig verwendet werden, dass die sozusagen "vorgefertigt" werden.

Links sehen wir das Symbol für unser XOR Gatter, sowie die Wahrheitstabelle zur Erinnerung (a XOR b ist genau dann 1, wenn entweder a oder b eine 1 repräsentiert).

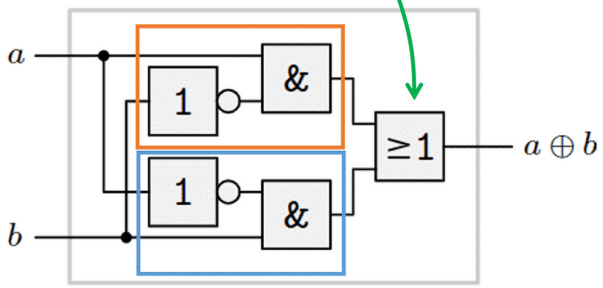
Auf der rechten Seite sehen wir die interne Implementierung der Schnittstelle. Oben betrachen wir aus welchen mathematischen Elementen das XOR tatsächlich bestehen.

Darunter sehen wir ein komplexes logisches Gatternetz, welches die logischen Operatoren zu Gatternetzen umwandelt und diese verbindet. Der graue Kasten symbolisiert, dass diese Schaltung in dem links oben dargestellten Gatter steckt. Da diese Funktionalität jedoch häufig gebraucht wird und dies aufwändig zu notieren ist, wird das XOR kompakt dargestellt. Dies gilt ebenfalls für alle weiteren Gatter die nicht elementar sind und oben notiert waren.



$$a \oplus b \equiv (a \wedge \neg b) \vee (\neg a \wedge b)$$

a	b	$a \oplus b$
0	0	0
1	0	1
0	1	1
1	1	0

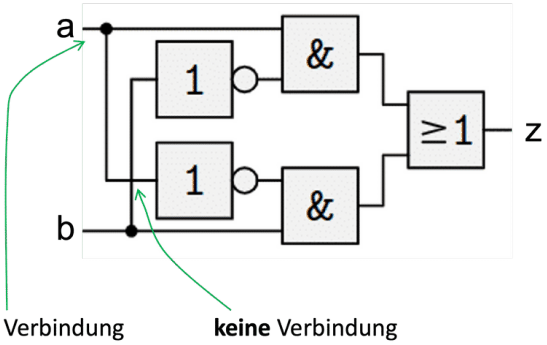


XOR: Gattersymbol, Wahrheitstabelle, logischer Ausdruck und Implementation aus elementaren Gattern.

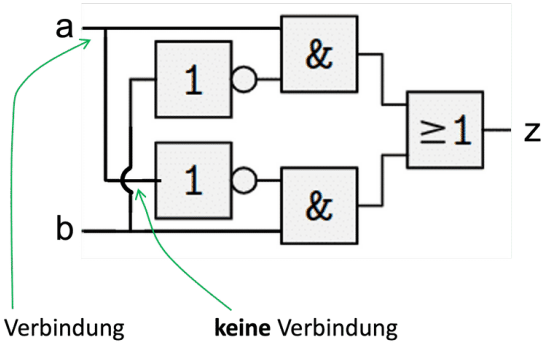
In dem obigen Bild sehen wir schwarze Punkte auf den Linien um Verzweigungen mit Stromflüsse zu symbolisieren. Eine Kreuzung zweier Linien ohne schwarzen Punkt bedeutet, kein Zusammenhang zwischen diesen beiden Elementen. Eine beliebte alternative Schreibweise ist die nicht zusammenhängigen Elementen mit einer "Brücke" einem kleinen Bogen bei der Kreuzung darzustellen. Hierbei wird dann der tatsächliche Zusammenhang ohne einen Punkt visualisiert.

Im Kontaktstudium verwenden wir Darstellungsmöglichkeit 1.

Darstellungsmöglichkeit 1



Darstellungsmöglichkeit 2



Darstellungsmöglichkeit für verschiedene Verbindungen zwischen den Linien.

Umwandlung einer Gleichung in eine Gatterschaltung

Schauen wir uns im Folgenden etwas genauer an, wie wir - ausgehend von einer Gleichung - eine Gatterschaltung entwerfen können. Wir nehmen dazu wieder unsere bekannte Gleichung

$$\boldsymbol{z} = (\neg\,(a \wedge b) \wedge \neg\,(a \vee c)) \wedge (\neg b \vee b)$$

und schauen uns zuerst den Prozess in einem Video und anschließend nochmals ausführlich in Einzelschritten an.

Zeichnen von Gatternetzen

$$z = \big(\neg(a \wedge b) \wedge \neg(a \vee c)\big) \wedge (\neg b \vee b)$$

Damit wir von einer Gleichung zu den Gatternetzen kommen starten wir mit den Variablen die Teil des Ausdrucks sind. Die Variablen werden als Eingänge auf die linke Seite geschrieben und der gesamte Ausdruck wird als z definiert. Dieser wird auf die rechte Seite geschrieben, zum symbolisieren des Ausganges.

$$\boldsymbol{z} = (\neg\,(a \wedge b) \wedge \neg\,(a \vee c)) \wedge (\neg b \vee b)$$

a
b

c

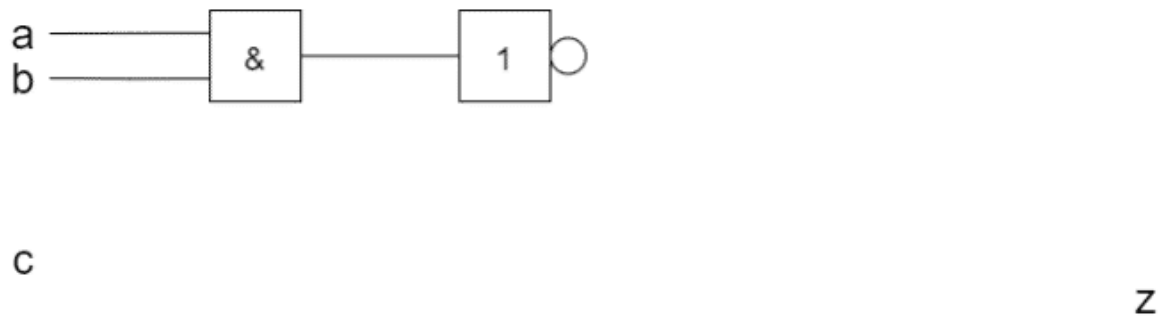
z

Als erstes Gatternetz starten wir mit dem Teilausdruck der am weitesten innen liegt. Da es zwei gibt, arbeiten wir den Ausdruck von vorne nach hinten ab. Wenn es mehrere innere Ausdrücke gibt ist es genau das gleiche wie zuvor, die Reihenfolge ist egal, nur die entsprechenden Regeln müssen beachtet werden.

$$\boldsymbol{z} = (\neg\,(a \wedge b) \wedge \neg\,(a \vee c)) \wedge (\neg b \vee b)$$

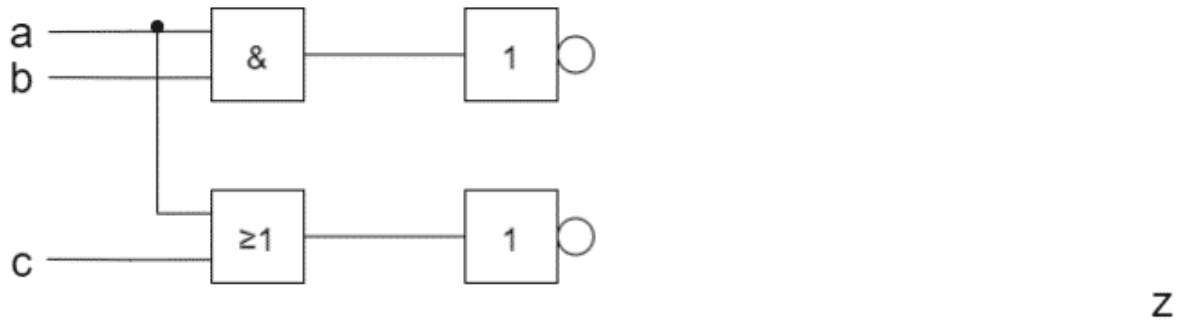
Nun betrachten wir die angehängte Negation. Dabei wird der bereits abgebildete Teilausdruck mit dem Umgebenden durch ein Gatter verknüpft.

$$z = (\neg\,(a \wedge b) \wedge \neg\,(a \vee c)) \wedge (\neg b \vee b)$$



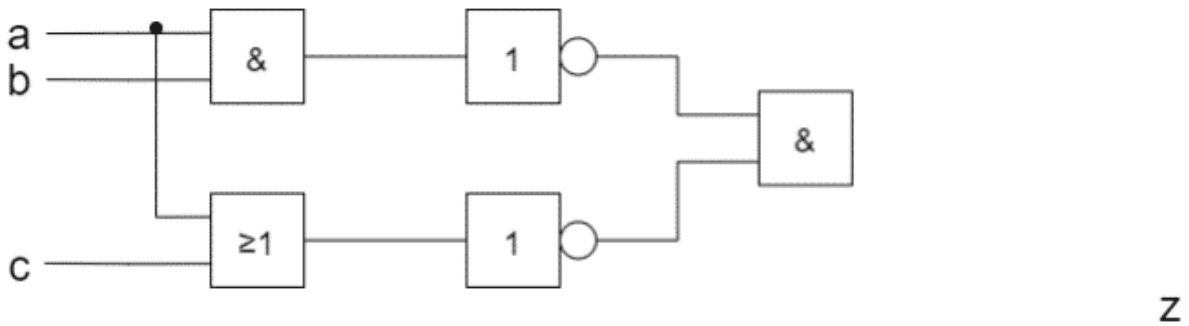
Als nächstes betrachten wir den zweiten inneren Ausdruck, sowie seine Negation.

$$z = (\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$$



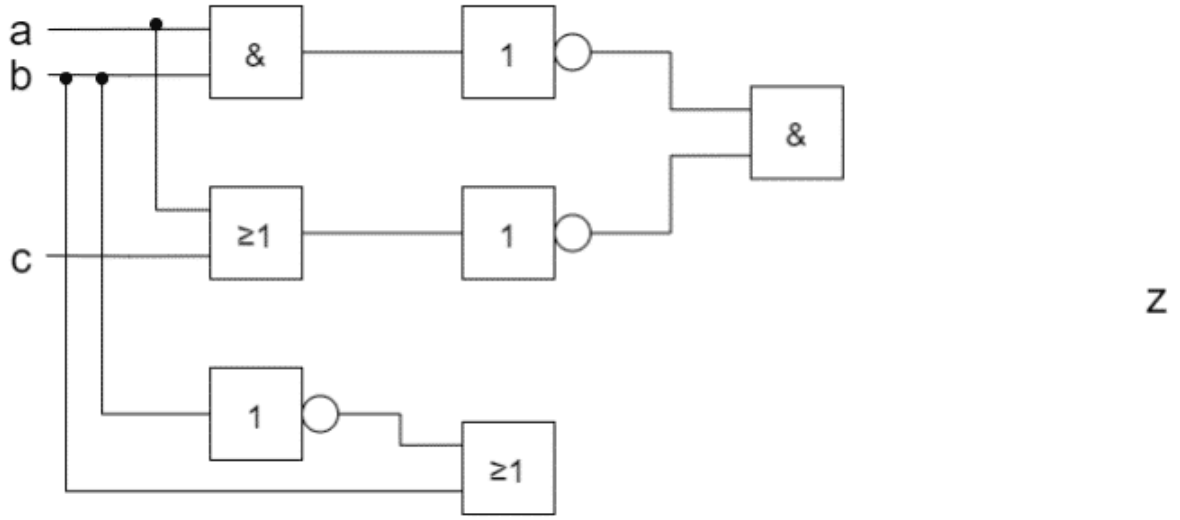
Nun wird der umgebende Teilausdruck hinzugefügt. Wir verbinden die beiden vorherigen Teilausdrücke.

$$z = (\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$$



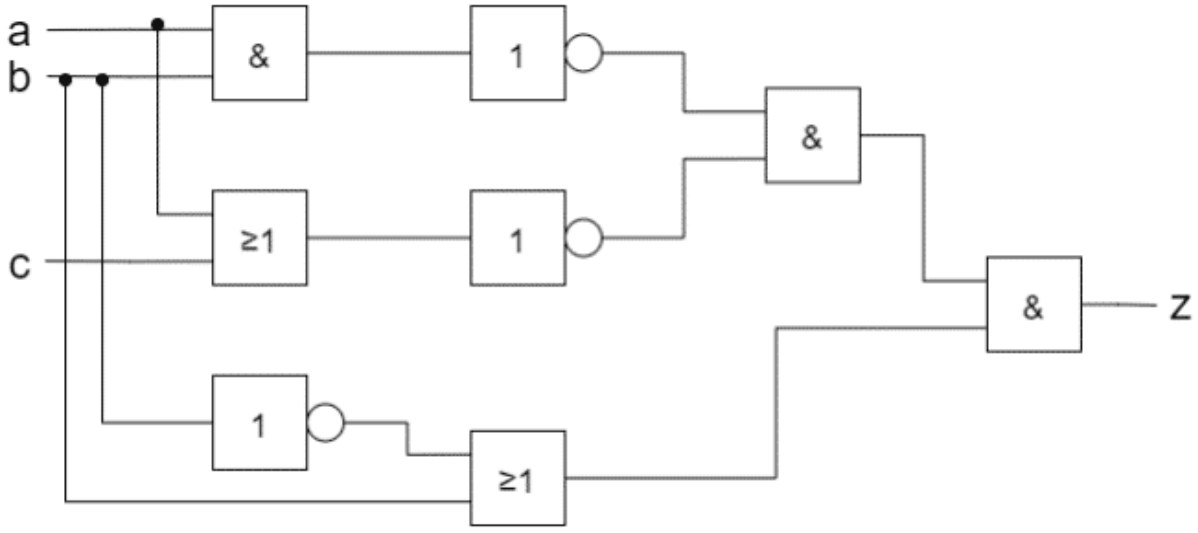
Als letztes fügen wir den hinteren Teil des Ausdruckes hinzu, er hat zum jetztigen Zeitpunkt keine Verbindung zu den restlichen Gattern.

$$z = (\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$$



Anschließend verbinden wir die beiden Teile mit einem Gatter.

$$z = (\neg (a \wedge b) \wedge \neg (a \vee c)) \wedge (\neg b \vee b)$$



Umwandlung zwischen Wahrheitstabelle und Gatterschaltung

Damit wir eine Wahrheitstabelle in eine Gatterschaltung überführen können benötigen wir den Zwischenschritt über eine Gleichung. Im Folgenden schauen wir uns dazu ein Beispiel an:

Zeichnen von Gatternetzen

a	b	c	…	$t = v \wedge u$
0	0	0	…	1
0	0	1	…	0
0	1	0	…	1
0	1	1	…	0
1	0	0	…	0
1	0	1	…	0
1	1	0	…	0
1	1	1	…	0

Axiomensystem - "Rechenregeln" für logische Ausdrücke

Wir haben bisher Wahrheitstabellen verwendet um logische Ausdrücke besser zu verstehen und zu vereinfachen. Wie bereits angesprochen, gibt es aber auch "Rechenregeln", die wir nutzen können um komplexe logische Ausdrücke zu vereinfachen. Einige dieser Regeln kennen Sie vermutlich bereits aus der Arithmetik; z.B. die Kommutativität oder die Distributivität.

Die "Rechenregeln" sind im Axiomsystem nach Peano festgeschrieben. Sie müssen diese Regeln nicht auswendig lernen. Falls wir Ihnen jedoch die Liste mit Regeln zur Verfügung stellen, müssen Sie die Regeln anwenden können und damit einen logischen Ausdruck vereinfachen können.

Axiomsystem nach Peano

Kommutativität	$a \wedge b = b \wedge a$	$a \vee b = b \vee a$
Assoziativität	$(a \wedge b) \wedge c = a \wedge (b \wedge c)$	$(a \vee b) \vee c = a \vee (b \vee c)$
Idempotenz	$a \wedge a = a$	$a \vee a = a$
Distributivität	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$

Neutralität	$a \wedge 1 = a$	$a \vee 0 = a$
Extremalität	$a \wedge 0 = 0$	$a \vee 1 = 1$
Involution	$\neg\neg a = a$	
De Morgan	$\neg(a \wedge b) = \neg a \vee \neg b$	$\neg(a \vee b) = \neg a \wedge \neg b$
Komplementarität	$a \wedge \neg a = 0$	$a \vee \neg a = 1$
Dualität	$\neg 0 = 1$	$\neg 1 = 0$
Absorption	$a \vee (a \wedge b) = a$	$a \wedge (a \vee b) = a$

Eine detaillierte Beschreibung der einzelnen Axiome sowie Hintergrundinformationen finden Sie in folgendem Video. Die logische Operator AND wird dabei als Konjuktion und die OR als Disjunktion bezeichnet. Diese Begriffe haben wir bisher nicht verwendet - dise sind aber gängige Begriffe für die entsprechenden logischen Operatoren.

Boolesche Algebra

Zusammenfassung

Zu Beginn der Lerneinheit C1 haben wir die drei Lernziele der Einheit vorgestellt:

- Sie können die wichtigsten **Begriffe zur Rechnerarchitektur** erklären und in Kontext zueinander bringen
- Sie können die **grundlegenden Darstellungsformen** (Wahrheitstabellen, Logische Ausdrücke und Gatterschaltungen) **erstellen**
- Sie können die **grundlegenden Darstellungsformen vereinfachen** und **ineinander überführen**

Nachdem Sie nun die Lerneinheit (fertig) bearbeitet haben, sollten Ihnen daher die folgenden Begriffe und deren Notationen vertraut sein:

- Schaltkreis / Schaltplan
- Parallelschaltung / Serienschaltung und Reihenschaltung
- Logische Operatoren
  AND / OR / NOT / XOR
- Schaltalgebra / boolesche Algebra / boolesche Logik
- Logische Ausdrücke und Gleichungen
  Operatorrangfolge und Klammern

- Variablen
- Wahrheitstabelle / Wahrheitstafel / Schalttabelle / Schalttafel
- Gatter / Gatternetz / Gatterschaltung
  Elementare Gatter und Zusammengesetze Gatter

Darüber hinaus sollten Sie in der Lage sein die grundlegenden Darstellungsformen

- **Logischer Ausdruck**
- **Wahrheitstabelle**
- **Gatterschaltung**

zu erstellen ...

... und ebenfalls die folgenden Darstellungsformen ineinander zu überführen:

- **Logischer Ausdruck -> Wahrheitstabelle**
- **Wahrheitstabelle -> logischen Ausdruck** (über disjunktive Normalform)
- **Logischer Ausdruck -> Gatterschaltung**
- **Gatterschaltung -> logischen Ausdruck**
- **Wahrheitstabelle -> Gatterschaltung**
  Zwischenschritt über einen logischen Ausdruck
- **Gatterschaltung -> Wahrheitstabelle**
  ebenfalls über den Zwischenritt eines logischen Ausdrucks

Schauen Sie doch nochmals in die Materialien, falls es hierbei noch Schwierigkeiten gibt.

Jetzt wünschen wir Ihnen viel Spass mit den Übungen!